

Eric Wentz
CV Basketball Pipeline Overview

Architecture:

My end-to-end computer vision basketball system transforms raw broadcast or single-camera footage into structured, court-relative player and event data. Each stage consumes the output of the one before it, moving from pixels, to detected objects, to tracked identities, to team-labeled, spatially-grounded, event-annotated player data.

Stage 1: Video Ingestion: The pipeline decodes a raw video clip into a stream of individual frames using the **supervision** library (OpenCV under the hood), pulling frames one at a time to keep memory usage manageable. Every later stage operates on this frame-by-frame basis.

Stage 2: Object Detection: Each frame is passed through an object detection model, which outputs a bounding box, class label, and confidence score for every relevant entity. Our model uses ten classes including possession- and action-specific classes like player-in-possession, player-jump-shot, ball-in-basket, and number (jersey number regions). Detection has no memory across frames, so the same player in frame 10 and frame 11 are unrelated until tracking connects them.

Stage 3: Tracking: This stage assigns each player a consistent ID that persists across frames, turning per-frame detections into continuous player trajectories. New detections are associated with existing tracks based on bounding box overlap, depending on the method used. The output is the same detections, now carrying a track ID, in MOT format (frame, track ID, bounding box). Which specific tracking method we use, and the tradeoffs between approaches, is discussed in the Model Choices section below.

Stage 4: Team Classification & Jersey Number: Once players have consistent track IDs, this stage assigns each one a team and, where visible, a jersey number. Each player crop is converted into a visual embedding capturing jersey color and pattern, reduced via UMAP, and clustered into two groups via K-means, fully unsupervised, meaning no labeled examples or prior knowledge of either team's colors is needed. Separately, the dedicated number class from detection localizes jersey number regions, which an OCR step reads into digits. The output is a team label and, where legible, a jersey number, per track.

Stage 5: Court Keypoints & Homography: This stage detects court landmarks, baselines, the three-point arc, and lane lines, as pixel coordinates with confidence scores. Once at least four landmarks are detected, they are matched to a standardized 2D court template to compute a homography: a transformation mapping any camera pixel to a court-space coordinate. The output combines track identity, team, and true court position per frame.

Stage 6: Shot & Event Detection: Stage 2's action-specific classes, player-jump-shot, player-in-possession, ball-in-basket, and similar, give this stage a direct signal for basketball events rather than requiring it to infer them from scratch. This stage watches for these classes to appear in sequence, for example a jump-shot detection followed by either a ball-in-basket detection or the ball leaving the frame near the rim, and combines that pattern with the shooter's tracked identity and court position to construct a structured event: shooter, team, shot type, location, and outcome.

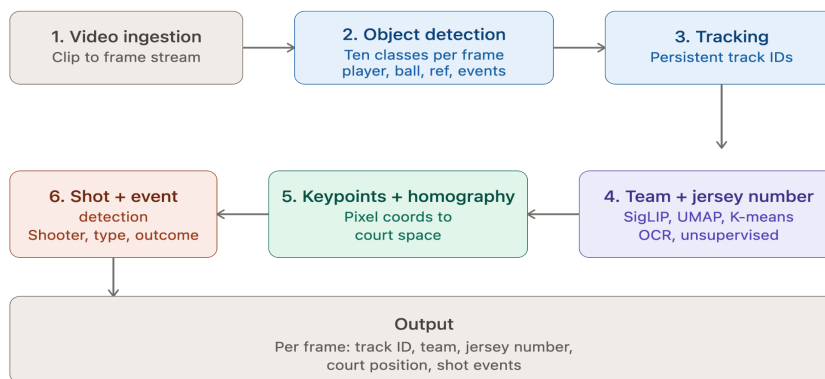


Figure 1. The six-stage pipeline, from raw video input to structured per-frame output including track identity, team, jersey number, court position, and shot events.

Model Choices

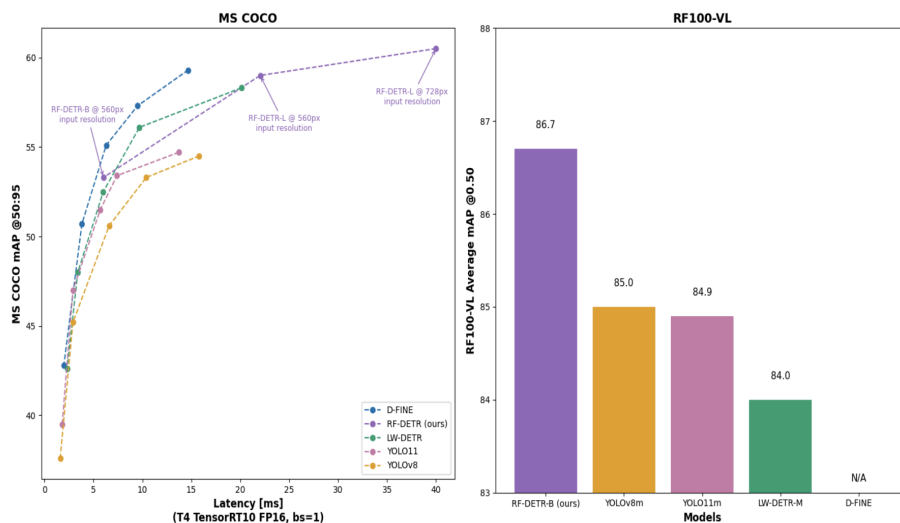
Object Detection: Model selection for object detection is extremely important as it is the literal foundation of our pipeline: we need to correctly identify what we are looking at before any downstream stage can do anything useful. During my capstone project for an introductory AI course, I was attacking a similar problem but did no model research. I just wanted to see what I could get working, so I used a YOLO-family model because it was readily available. At the time I did not know the difference between approaches, and looking back that was a meaningful gap.

YOLO is a convolutional neural network (CNN) (Redmon et al., 2016). The way to think about it is like a magnifying glass sliding across the pixels of a still frame, picking out edges, colors, and patterns in small, isolated patches. It is fast and effective, but each patch is processed independently, without any awareness of what is happening in another region of the same image. YOLO models perform well on COCO, the canonical 163k image benchmark from Microsoft (Lin et al., 2014), but this is increasingly a limitation rather than a credential. As Robinson et al. (2025) note, object detection models face a kind of evaluation crisis: COCO has not kept pace with how computer vision is actually being applied, and models that maximize COCO performance are implicitly tuning themselves to a narrow slice of the real world.

The alternative we chose is RF-DETR, Roboflow's real-time detection transformer (Robinson et al., 2025; Roboflow, 2025). Transformers were introduced to real-time object

detection through models like RT-DETR (Zhao et al., 2024), and use a self-attention mechanism that allows every region of a single frame to communicate with every other region simultaneously, rather than processing isolated patches independently

Figure 2.
(Robinson et al., 2025).
**RF-DETR is the first
realtime model to
achieve 60+ mAP on
COCO. Performance
is comparable to
current
state-of-the-art at
lower latencies.**



The evidence for this choice is in the RF100-VL accuracy-latency Pareto curve (Figure 1, Robinson et al., 2025), a benchmark of 100 real-world datasets specifically designed to test domain adaptability rather than COCO performance (Robicheaux et al., 2025). RF-DETR forms the best Pareto frontier: highest accuracy at any given latency, and the curve continues improving at scale in a way YOLO variants do not. D-FINE (Peng et al., 2024), another strong transformer-based detector, is notably absent from this comparison entirely: its authors do not support reliable fine-tuning, meaning its real-world adaptability cannot be measured (Robinson et al., 2025). For a basketball-specific application where we need the model to adapt to a narrow visual domain, an untested model is no model at all.

Tracking: We selected ByteTrack (Zhang et al., 2022), an IoU-based tracker built directly into the supervision library. Its key differentiator is that it retains low-confidence detections as tentative tracks rather than discarding them, handling partially occluded or briefly out-of-view players more gracefully than naive IoU matching. Our fixed camera setup also makes it particularly viable, since IoU-based association degrades primarily when the camera itself is moving.

We considered three alternatives. SAM2 offers pixel-level segmentation tracking and would be the most accurate option, but requires CUDA and was eliminated for development compatibility. DeepSORT (Wojke et al., 2017) adds appearance embeddings and a Kalman filter on top of IoU matching but requires a separate re-identification model and is slower, overkill for

a fixed camera with visually distinct jersey colors. BoT-SORT (Aharon et al., 2022) combines motion, appearance, and camera motion compensation and is arguably the strongest box-based tracker, but the camera motion compensation is unnecessary for a fixed tripod setup and the added complexity is not justified at this scope.

Team Classification & Jersey Number (OCR):

Team classification uses the unsupervised SigLIP embedding pipeline described in the Architecture section, requiring no labeled jersey data. Model selection here is straightforward: SigLIP was chosen for its strong visual feature extraction and zero-shot generalization, and K-means with $k=2$ maps cleanly onto the two-team structure of basketball without supervision.

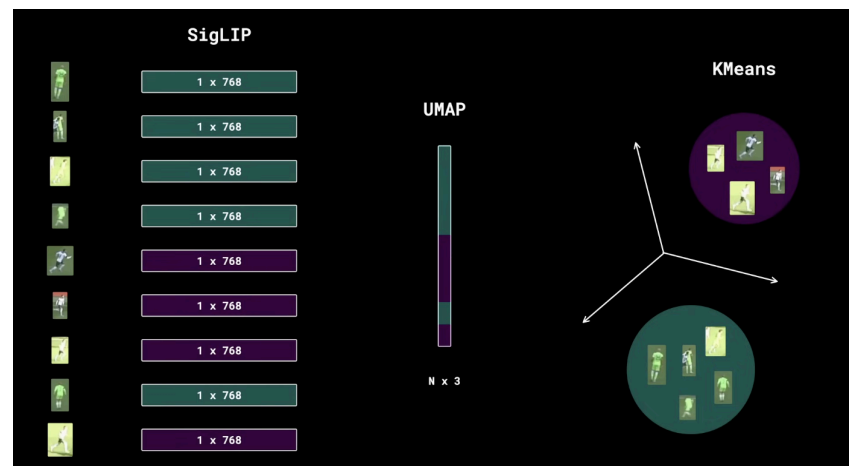


Figure 3. (Roboflow) Using SigLIP, UMAP, and Kmeans

For jersey number recognition, we selected SmolVLM2, a vision-language model from HuggingFace that takes a cropped jersey region and responds to the prompt "Read the number." in natural language (Roboflow, 2025). It requires no labeled training data and works zero-shot out of the box, with a reported base accuracy of roughly 56% and fine-tuned accuracy of roughly 86%. The main alternative is a ResNet-based classifier trained on labeled jersey crops, which Roboflow benchmarks at 93% accuracy and faster inference, but requires labeled training data per jersey style. Traditional OCR tools like EasyOCR and Tesseract were ruled out early: they are not designed for the stylized, curved, or motion-affected numbers common in sports footage.

In practice, SmolVLM2 never ran on my footage. RF-DETR's number class produced minimal detections even at a confidence threshold of 0.1, meaning there were no crops to feed into the OCR pipeline. The likely root cause is a domain mismatch: the detection model was probably fine-tuned on footage with different jersey styles and camera angles than our clips, and motion blur on number regions further suppresses confidence scores. This is an honest failure mode worth noting: the OCR architecture is sound, but it is gated entirely on upstream detection quality, and that gate did not open on our test footage. A ResNet classifier trained specifically on our jersey style would be the production fix, combined with a detection model with stronger number class coverage.

Court Keypoints & Homography: For court landmark detection we used a YOLO-pose variant, chosen primarily because it worked: during my capstone project I trained it on approximately 100 labeled images of basketball court keypoints across half-court and full-court configurations and achieved 99.5% mAP@50, 99.7% precision, 100% recall, and 99.8% F1 (Wentz, 2026). YOLO-pose is a single-stage model, fast and CPU compatible, and basketball courts are visually standardized enough that a lightweight model handles the task without needing a heavier architecture. The main alternative is ViTPose (Xu et al., 2022), a transformer-based keypoint model that is state of the art on the COCO keypoint benchmark and would likely offer marginal accuracy improvements, particularly on partially occluded landmarks when players crowd the paint.

Shot & Event Detection: No additional model is required for this stage. As described in the Architecture section, our RF-DETR detection model already includes action and event-specific classes: player-in-possession, player-jump-shot, player-layup-dunk, player-shot-block, and ball-in-basket. Shot and event detection is therefore handled by the same model driving Stage 2, with this stage responsible for interpreting the temporal pattern of those class labels across frames rather than running a separate model. This is a deliberate design choice: by selecting a detection model with a rich enough class taxonomy upfront, we avoid the need for a dedicated event classification model entirely.

Data Strategy

For a pipeline like this, the data is almost entirely human in the loop: someone has to watch basketball footage, pause on frames, and draw boxes or place points. There is no shortcut, and having manually labeled hundreds of images myself during my capstone, I know firsthand what that costs in time and effort.

The primary source is basketball footage of any kind: YouTube clips, college game film, or publicly available datasets like SportsMOT. Footage is split into still frames, uploaded to Roboflow, and manually annotated with bounding boxes for each class: players, referees, ball, rim, and jersey number regions. The same footage then requires a second annotation pass for court keypoint detection, placing labeled points on landmarks like baselines, arcs, and lane intersections. Occluded landmarks need to be marked as occluded rather than skipped: if a player is standing on a lane corner, that point still exists and the model needs to know it.

Augmentation is where robustness comes from without additional labeling. Noise, motion blur, brightness shifts, and random crops force the model to handle degraded footage rather than memorizing clean frames.

Evaluation

Each stage is evaluated independently since no single metric captures the full pipeline.

Object Detection: mAP@50 (Mean Average Precision) is the standard metric, measuring detection accuracy at an IoU threshold of 0.5 (Lin et al., 2014). mAP@50-95 is stricter, averaging across IoU thresholds and better reflecting box quality. Per-class precision and recall are reported alongside overall mAP, particularly for the number class which failed entirely on our footage.

Tracking: MOTA is our primary metric, computed as one minus the sum of false positives, false negatives, and identity switches divided by total ground truth detections (Bernardin & Stiefelhausen, 2008). We chose MOTA for its simplicity and wide implementation. ID switch count is reported alongside it as the most direct signal of ByteTrack's core failure mode. HOTA, which weights detection and association quality equally, would be preferred in production (Luiten et al., 2021) and is a clear next step.

Team Classification: Accuracy is the primary metric, the percentage of player crops correctly assigned to their team, verified against manually labeled ground truth. Given the binary nature of the problem and the visual distinctiveness of jersey colors, accuracy is sufficient. A UMAP or t-SNE plot of the embedding space serves as a useful qualitative complement, showing whether the two clusters are cleanly separated. Our own embedding visualization showed clear separation between teams (Wentz, 2025).

Jersey OCR: Lock rate is the most important metric here: the percentage of players that received any number assignment at all. On our footage this was effectively zero, since RF-DETR's number class produced no detections to crop and feed into SmolVLM2. Top-1 accuracy, the percentage of exact digit matches, is reported for any crops where detection did succeed. Top-5 accuracy is not meaningful here — jersey number identification requires an exact match.

Court Keypoints: OKS-mAP (Object Keypoint Similarity) is the standard metric, measuring how close predicted keypoints are to ground truth weighted by joint importance (Lin et al., 2014). In practice our model achieved 99.5% mAP@50 on a held-out test set (Wentz, 2025).

Shot & Event Detection: Per-class F1 is the primary metric for each event type: jump-shot, layup-dunk, shot-block, and ball-in-basket. F1 balances precision and recall into a single score, which matters here because shot events are rare within a short clip and accuracy alone would be misleading.

Failure Modes & Risks

The most pervasive failure mode in this pipeline is video quality. The entire system is built on detection, and detection degrades with the footage. Motion blur, poor lighting, low resolution, and fast camera cuts all directly reduce confidence scores across every class. The ball is the most vulnerable object: it is small, fast moving, and frequently appears as a streak rather

than a distinct round object, causing it to drop below detection thresholds regularly. This is a known benchmark challenge in sports computer vision and not unique to our implementation (Huang et al., 2019).

False positives from fans and coaches near the sideline are a real and observed failure. From a distance, a coach in dark clothing standing near the baseline looks enough like a player that the model will box them. This is a data problem as much as a model problem: negative examples of non-players near the court need to be well represented in training data.

Homography is fragile when court landmarks are occluded. When players crowd the paint, the keypoints the homography depends on disappear, causing player positions on the 2D court to drift or fail entirely until enough landmarks become visible again. Camera cuts and angle changes compound this: when the footage jumps to a different view, the homography has to recompute from scratch and tracking loses all existing IDs.

Team classification via SigLIP and K-means breaks down when both teams wear similarly colored jerseys. The two-cluster assumption holds when home and away jerseys are visually distinct, but fails in matchups with similar palettes, and the model has no fallback.

Compute & Latency

Training is the expensive part, both in human time and compute time. During my capstone, labeling alone took hours before a single training run began, and the actual training added to that. We experimented with image size, batch size, and epoch count across eight iterations of the object detection model and six for keypoints, each run taking several hours on a GPU. Environment setup, dependency conflicts, and GPU mismatches added overhead that is easy to underestimate going in.

Inference is significantly lighter. RF-DETR-B runs at roughly 4-5ms per frame on a GPU, and ByteTrack adds negligible overhead on top. On CPU, inference slows considerably but remains viable for batch processing of pre-recorded footage.

The system as designed is batch, not real-time. A coach uploads a clip and gets results back in minutes, which fits the use case. A production deployment with a dedicated GPU and optimized inference could push toward near-real-time, but real-time live broadcast processing remains a longer-term goal.

References

- Aharon, N., Orfaig, R., & Bobrovsky, B. Z. (2022). BoT-SORT: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*.
- Bernardin, K., & Stiefelhausen, R. (2008). Evaluating multiple object tracking performance: The CLEAR MOT metrics. *EURASIP Journal on Image and Video Processing*, 2008, 1–10.
- Cui, Y., Zeng, C., Zhao, X., Yang, Y., Wu, G., & Wang, L. (2023). SportsMOT: A large multi-object tracking dataset in multiple sports scenes. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 9921–9931.
- Huang, Y.-C., Liao, I.-N., Chen, C.-H., Ik, T.-U., & Peng, W.-C. (2019). TrackNet: A deep learning network for tracking high-speed and tiny objects in sports applications. *arXiv preprint arXiv:1907.03698*.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. *European Conference on Computer Vision (ECCV)*, 740–755.
- Luiten, J., Osep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., & Leibe, B. (2021). HOTA: A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2), 548–578.
- Peng, Y., Li, H., Wu, P., Zhang, Y., Sun, X., & Wu, F. (2024). D-FINE: Redefine regression task in DETRs as fine-grained distribution refinement. *arXiv preprint arXiv:2410.13842*.
- Robicheaux, P., Popov, M., Madan, A., Robinson, I., Nelson, J., Ramanan, D., & Peri, N. (2025). Roboflow100-VL: A multi-domain object detection benchmark for vision-language models. *arXiv preprint arXiv:2505.20612*.
- Robinson, I., Robicheaux, P., Popov, M., Ramanan, D., & Peri, N. (2025). RF-DETR: Neural architecture search for real-time detection transformers. *arXiv preprint arXiv:2511.09554*.
- Roboflow. (2025). RF-DETR: A SOTA real-time object detection model. *Roboflow Blog*. <https://blog.roboflow.com/rf-detr/>
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.
- Wentz, E. (2025). Computer vision on basketball film report. *COMP 484 Capstone, Macalester College*.

Wojke, N., Bewley, A., & Paulus, D. (2017). Simple online and realtime tracking with a deep association metric. *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 3645–3649.

Xu, Y., Zhang, J., Zhang, Q., & Tao, D. (2022). ViTPose: Simple vision transformer baselines for human pose estimation. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 38571–38584.

Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. *European Conference on Computer Vision (ECCV)*, 1–21.

Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., & Chen, J. (2024). DETRs beat YOLOs on real-time object detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16965–16974.

Zhai, X., Mustafa, B., Kolesnikov, A., & Beyer, L. (2023). Sigmoid loss for language image pre-training. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 11975–11986.